

ゼロから python で行列の簡約化

MMA 平部員 B1 Tenpura

2026 年 9 月 20 日

1 はじめに

こんにちは。B1 平部員の Tenpura です。この記事では夏休みに重い腰を上げて知識ゼロから Python で行列式を書いてみた話をします。動機は Anki のアドオン作りで Python わからんことによる萎え落ちを減らすことです。あと線形代数で追試くらったことです。雪辱を果たしに。

2 行列とは

行列は縦横に数を並べたものです。例えば
$$\begin{cases} x + 2y - z = 2 \\ 2x - y + 3z = 9 \\ 3x + y + z = 8 \end{cases} \quad \text{を} \quad \begin{bmatrix} 1 & 2 & -1 & 2 \\ 2 & -1 & 3 & 9 \\ 3 & 1 & 1 & 8 \end{bmatrix}$$

のように書けます。シンプルですね。

3 簡約化とは

簡約化は、連立方程式を解く際に行列の値を都合よくする操作です。実際中学からの連立方程式の解き方まんまで、式同士を足し引きします。例えば、3つの式を上から①②③として、

$$\begin{aligned} & \begin{bmatrix} 1 & 2 & -1 & 2 \\ 2 & -1 & 3 & 9 \\ 3 & 1 & 1 & 8 \end{bmatrix} \xrightarrow[\text{③}-3\times\text{①}]{\text{②}-2\times\text{①}} \begin{bmatrix} 1 & 2 & -1 & 2 \\ 0 & -5 & 5 & 5 \\ 0 & -5 & 4 & 2 \end{bmatrix} \xrightarrow{\text{②}\times(-\frac{1}{5})} \begin{bmatrix} 1 & 2 & -1 & 2 \\ 0 & 1 & -1 & -1 \\ 0 & -5 & 4 & 2 \end{bmatrix} \\ & \xrightarrow[\text{③}+5\times\text{②}]{\text{①}-2\times\text{②}} \begin{bmatrix} 1 & 0 & 1 & 4 \\ 0 & 1 & -1 & -1 \\ 0 & 0 & -1 & -3 \end{bmatrix} \xrightarrow{\text{③}\times(-1)} \begin{bmatrix} 1 & 0 & 1 & 4 \\ 0 & 1 & -1 & -1 \\ 0 & 0 & 1 & 3 \end{bmatrix} \xrightarrow[\text{②}+\text{③}]{\text{①}-\text{③}} \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 3 \end{bmatrix} \end{aligned}$$

と変形します（行基本変形と言います）。最後の行列は

$$\begin{cases} x + 0 + 0 = 1 \\ 0 + y + 0 = 2 \\ 0 + 0 + z = 3 \end{cases}$$

となり、良い感じに文字の値が分かります。この操作を言語化すると、

-
1. すべての成分が 0 でない列のうち、一番左の列に注目する。
 2. 注目した列の一番上の数が 0 のときは、0 でない行と入れ替える。
 3. 第 1 行を何倍かして主成分をつくる。
 4. 第 1 行を何倍かして他の行に加えて、主成分の上下にある成分をすべて 0 にする。
 5. 第 1 行を忘れ、残った行で 1~4 を繰り返す。
-

となります。¹⁾我々 B1 は 4 月の不安な時期にこれを頭に叩き込んだと思います。
個人的な話ですが 4 行 4 列よりサイズが大きくなると必ず計算ミスをします。どうにかしたい。

4 計画

今回はこれを Python で実装します。しかし、上記の説明のまま実装しようとする割と苦労します。例えば 5. で繰り返しの操作がありますが、コードで書くには行数を文字で置いたりして一般化する必要があります。そこで、以下のように考えてみます。

行列を縦 I 、横 J の大きさとする。はじめ、 $k=1$ とする。

1. 左から右の列へ、各列で k 行目から順に 0 でない数を探す。はじめに見つかる数を n とする。
 2. n がある行を k 行目と入れ替える。
 3. k 行目を n で割る。
 4. k 行目以外の行から k 行目を何倍かして引き、 n と同じ列の n 以外の数を 0 にする。
 5. k に 1 を加え、1. からやり直す。1. で $k>I$ となるか、 J 列目を超えたとき、計算を終了する。
-

使いやすい文字をいくつか入れてみました。いっぺんに実装を考えるときついです、順番ごとに考えるとなんかいけそうな気がします。

1) 培風館 線形代数学講義 [増補版] 木田雅成 著 P40 III.7 より引用

5 Python のお約束

知識ゼロから Python を書くにあたって、Python のお約束を知っておこう。Python を使う準備や基本構文は Python 公式入門講座²⁾が簡単に教えてくれます。コンリテの数倍分かりますので気楽にやりましょう。この章では補足的に、行列の書き方と「おまじない」を紹介します。

5.1 リスト内包表記

Python で行列を扱う際、リスト内包表記が便利です。リスト³⁾の中にリストを入れる書き方です。例えば $\begin{bmatrix} 1 & 2 & -1 & 2 \\ 2 & -1 & 3 & 9 \\ 3 & 1 & 1 & 8 \end{bmatrix}$ を $A = [[1, 2, -1, 2], [2, -1, 3, 9], [3, 1, 1, 8]]$ と書きます。

改行は無くてもよく、一行に直すと $A = [[1, 2, -1, 2], [2, -1, 3, 9], [3, 1, 1, 8]]$ となります。

これを使い倒すなら行列の大きさや一つ一つの数が変数に欲しい所です。以下に実装に使うものをまとめます。

表 1: リスト内包表記の行列	
a_{ij} (i 行 j 列目の数)	<code>a[i-1][j-1]</code>
行数	<code>len(a)</code>
列数	<code>len(a[0])</code>

注意！！ Python のリストは順番を 0 から数えます。例えば a_{23} は `a[1][2]` です。この数え方は大体のプログラミング言語で共通し、度々間違えてバグの温床となります。私は現在進行形で間違えます。記事のどこかには絶対間違ってますが許して〜〜

5.2 numpy,simpy

他には numpy や simpy というライブラリ⁴⁾で行列を扱えます。こちらは簡約化、行列式の計算、単位行列、ゼロ行列などのプリセットが入っていて、行列でやりたい操作は大体何でもできます。例えば今回実装する簡約化は A を行列として `A.rref()` で終わります。これで終わるのはさすがにおもんなさすぎるので、今回はリスト内包表記で頑張ります。

2) <https://www.python.jp/train/index.html>

3) 複数の要素をまとめて扱う形式。日常の「リスト」と同じ。

4) 便利機能が集まっている工具箱

5.3 おまじない

我々初心者がわけもわからずファイル先頭に入れさせられる謎の文字列こと、おまじない。正体は $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ の `\usepackage{}` と同じく、使うライブラリを書いておく部分です。先ほどの `numpy` などマジで便利なものがあるのでよく弄りましょう！！

ただ、今回は実装を楽しみたいので以下の基本的なものだけを入れます。

コード 1: 便利なおまじない

```
1 import re
2 from decimal import Decimal
3 from fractions import Fraction
```

「`re`」は正規表現、「`Decimal`」は正確な小数計算、「`Fraction`」は分数を扱います。どれも便利。

6 実装

ようやく実装に入ります。4章に従ってコードを書いていきます。

テストの際おまじないを忘れないでください。また、使う行列は $A = \begin{bmatrix} 1 & 2 & -1 & 2 \\ 2 & -1 & 3 & 9 \\ 3 & 1 & 1 & 8 \end{bmatrix}$ として、以下のように実装していきます。

コード 2: 概形

```
1 import re
2 from decimal import Decimal
3 from fractions import Fraction
4
5 A = [[1, 2, -1, 2], [2, -1, 3, 9], [3, 1, 1, 8]]
6
7 def simplify(a):
8     k = 0
9     while k < len(a):
10         [1つ目の処理]
11         [2つ目の処理]
12         [3つ目の処理]
13         [4つ目の処理]
14     return a
15
16 def output(a):
17     [見た目の処理]
18
19 print("計算結果:")
20 output(simplify(A))
```

コード 3: 左から右の列へ、各列で k 行目から順に 0 でない数を探す。はじめに見つかる数を n とする。

```
1 def simplify(a):
2     k = 0
3     while k < len(a):
4         i = process_row
5         j = 0
6         while j < len(a[0]) and a[i][j] == 0:
7             if i == len(a)-1:
8                 i = k
9                 j = j + 1
10            else:
11                i = i + 1
12        if j >= len(a[0]):
13            break
14        print(f"a{i+1,j+1}に注目")
15        tmp_i = i
16        tmp_j = j
```

プログラムのソースコードをレポートに載せたいときは、listings パッケージを利用して lstlisting 環境で囲むと良いだろう。例えば、コード 4 のように書くと Python のコードがきれいに表示される。

コード 4: 便利なおまじない

```
1 def hello():
2     print("Hello, World!")
3
4 if __name__ == "__main__":
5     hello() # この行はコメントです
```

6.1 参考文献の明示

レポートを書く上では他の文献から情報を得ることも多いであろう。こういう時は、thebibliography 環境に参考文献として書籍やネットの記事を並べて出典を明確にするべきである。ネット上の情報の引用の際には \url マクロを利用して <https://www.example.com/> のように書くが良いだろう [1]。

参考文献を書くときは、\bibitem マクロを用いて文献を列挙する。このマクロに指定したラベルを \cite マクロで参照することで、文献番号を出力できる。

参考文献

- [1] 奥村晴彦, 黒木裕介, [改訂第 8 版] LaTeX2 ε 美文書作成入門, 技術評論社, 2020.

付録 A: 付録の書き方

本文に書くべきではない細かい計算や、導出は付録として別個にまとめることが多い。付録に入る際は`\appendix` マクロを用いる。その後、附番はアルファベットになり表番号や図番号の表記も変更される。なお、`\appendix` は1度書いた後はもう書く必要はない。